STUDENT RESULT MANAGEMENT SYSTEM

PROJECT REPORT

Abdullah Nasir (CT-25076)

M. Saad Ahmed (CT-25089)

Faeez Ur Rehman (CT-25090)

Instructor: Sir Abdullah

Programming Fundamentals

CT-175

Table of Contents

Programming Fundamentals

CT-175

PROJECT TITLE

STUDENT RESULT MANAGEMENT SYSTEM

ABSTRACT

This program is designed to generate a simple Student Result Management System using C language. It allows users to input a student's name, number of subjects, individual subject names, and obtained marks. The program calculates the percentage and corresponding grade for each subject, along with the overall percentage and overall grade. The output is presented in a clear tabular format, making it easy to interpret the student’s academic performance.

INTRODUCTION

Assessment of academic performance is essential in educational systems. Manually calculating grades for multiple subjects can be time-consuming and prone to error. This program automates the process by collecting marks of various subjects and computing result-related statistics. It uses basic programming constructs such as loops, arrays, and user-defined functions to evaluate student performance efficiently and systematically.

Programming Fundamental

CT-175

TECNICAL TERMS & CONCEPTS

Term / Concept	Description
Array	Used to store multiple values of the same type (e.g., subject names and marks).
Function (getGrade)	A user-defined function that determines the grade based on percentage.
Loop (for loop)	Repeatedly executes input and output operations for multiple subjects.
Percentage Calculation	(obtained marks / total marks) * 100 formula used per subject and overall.
Conditional Statements (if-else)	Used to assign grades based on defined percentage criteria.
Formatted Output (printf)	Used for structured and readable result display.

Programming Fundamentals

CT-175

UNDERSTANDING & USAGE OF CODE

The program collects the student’s name, number of subjects, subject names, and obtained marks. It uses arrays to store the subject data and loops to input multiple values efficiently. This allows the program to handle any number of subjects entered by the user.

For each subject, the program calculates percentage and determines the corresponding grade.The results are displayed in a table format to clearly show subject-wise performance. This tabular display helps in easily comparing marks and grades among subjects.

The getGrade() function is used to assign grades based on percentage values. This separates grade logic from the main program, making the code cleaner and reusable. It improves readability and makes future grading rule changes easier.

The program sums all marks to calculate the total obtained and overall percentage. Based on this percentage, an overall grade is computedand displayed. This provides a complete summary of the student'sacademic performance.

PROGRAM CODE

```c
#include <stdio.h>
#include <string.h>   // For using string length and related functions

// ---------------------------------------------------------------
// Function: getGrade
// Purpose : Determine grade based on percentage value
// Input   : float percentage
// Output  : grade character (A, B, C, D, F)
// ---------------------------------------------------------------
char getGrade(float percentage) {
    if (percentage >= 80)
        return 'A';
    else if (percentage >= 70)
        return 'B';
    else if (percentage >= 60)
        return 'C';
    else if (percentage >= 50)
        return 'D';
    else
        return 'F';
}

// ---------------------------------------------------------------
// Main Function
// ---------------------------------------------------------------
int main() {

    // ********** Variable Declarations **********
    char name[50];       // Stores student full name
    int subjects , i;        // Number of subjects student has

    // ********** Input Section **********
    printf("Enter Student Name: ");
    scanf("%[^\n]", name);   // Allows spaces in name
```

PROGRAM CODE

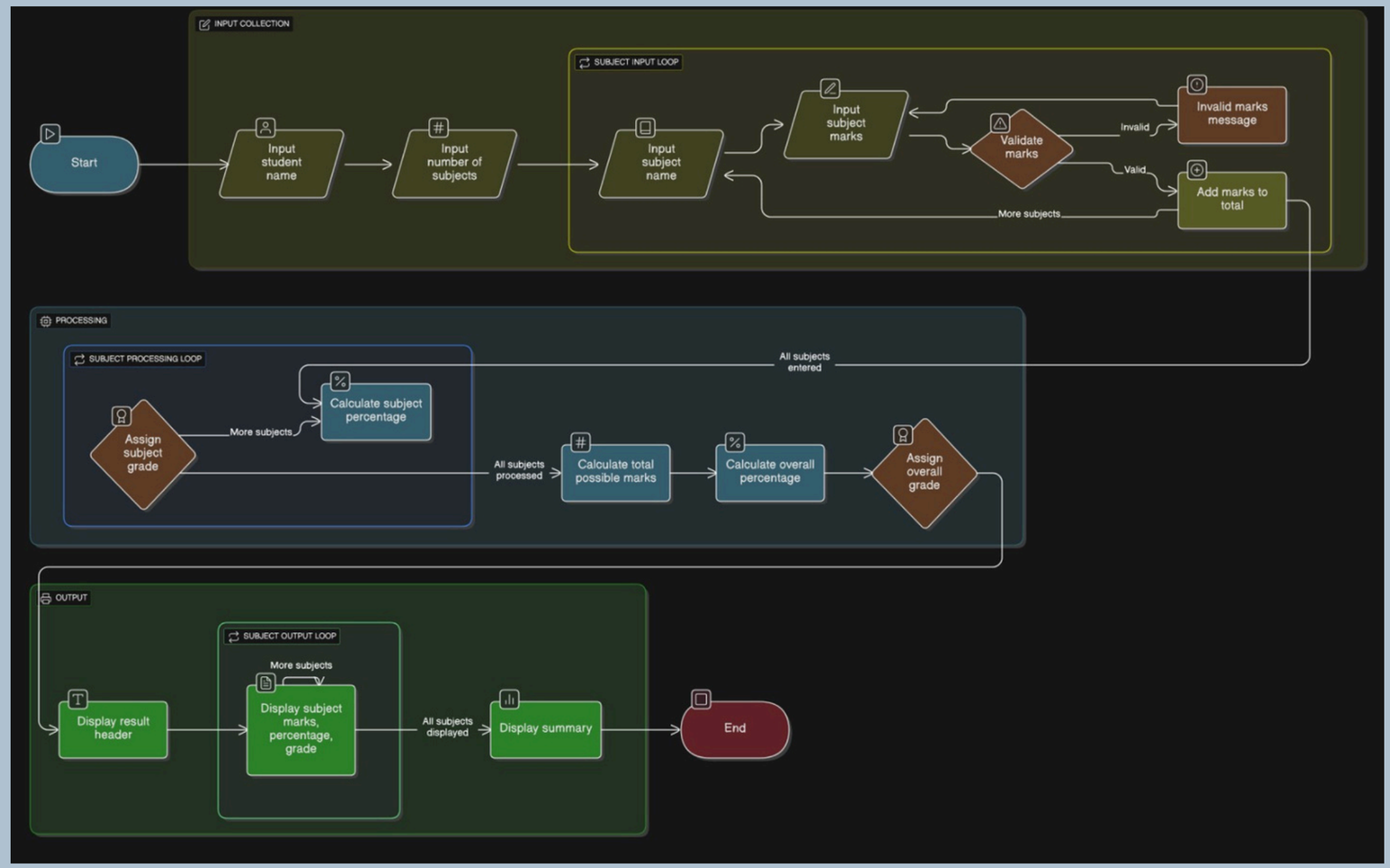
```c
printf("Enter number of subjects: ");
scanf("%d", &subjects);

// Declaring arrays after getting subjects count
char subName[subjects][20];  // Stores subject names
float marks[subjects];       // Stores marks of each subject

float maxMarks = 100;        // Maximum marks per subject
float obtainedTotal = 0;     // Sum of all obtained marks

// ********** Loop to Input Subject Names & Marks **********
for (i = 0; i < subjects; i++) {

    printf("\nEnter name of subject %d: ", i + 1);
    scanf("%s", subName[i]);   // Only single word names allowed

    printf("Enter marks in %s (out of 100): ", subName[i]);
    scanf("%f", &marks[i]);

    // Validating Marks (no negative or >100 allowed)
    if (marks[i] < 0 || marks[i] > 100) {
        printf("Invalid Marks! Please enter again.\n");
        i--;    // Repeat this iteration
        continue;
    }

    obtainedTotal += marks[i];  // Add to total obtained marks
}
```

```c
    // ********** Displaying Student Result Header **********
    printf("\n\n================ STUDENT RESULT ================\n");
    printf("Student Name: '%s'\n", name);
    printf("------------------------------------------------\n");
    printf("Subject\t\tMarks\tPercentage\tGrade\n");

    // ********** Loop to Display Individual Subject Records **********
    for (i = 0; i < subjects; i++) {

        float percent = (marks[i] / maxMarks) * 100;  // subject percentage
        char grade = getGrade(percent);               // grade for subject

        // Display each subject marks detail
        printf("%-10s\t%.2f\t%.2f%%\t\t%c\n", subName[i], marks[i], percent, grade);
    }

    // ********** Overall Result Calculation **********
    float totalPossibleMarks = subjects * maxMarks;              // Total marks possible
    float overallPercent = (obtainedTotal / totalPossibleMarks) * 100; // Overall Percentage
    char overallGrade = getGrade(overallPercent);           // Overall grade

    // ********** Display Final Summary **********
    printf("------------------------------------------------\n");
    printf("Total Obtained Marks: %.2f\n", obtainedTotal);
    printf("Total Subject marks:%.2f\n",totalPossibleMarks);
    printf("Overall Percentage: %.2f%%\n", overallPercent);
    printf("Overall Grade: '%c'\n", overallGrade);
    printf("================================================\n");

    // ********** End Program **********
    return 0;
}
```

Programming Fundamentals

CT-175

FLOWCHART



Programming Fundamentals

CT-175

CODE OUTPUT

```
Enter Student Name: Hasnain Khan
Enter number of subjects: 3

Enter name of subject 1: Maths
Enter marks in Maths (out of 100): 78

Enter name of subject 2: English
Enter marks in English (out of 100): 89

Enter name of subject 3: Urdu
Enter marks in Urdu (out of 100): 77


=============== STUDENT RESULT ================
Student Name: 'Hasnain Khan'
-------------------------------------------------
Subject         Marks   Percentage      Grade
Maths           78.00   78.00%          B
English         89.00   89.00%          A
Urdu            77.00   77.00%          B
-------------------------------------------------
Total Obtained Marks: 244.00
Total Subject marks:300.00
Overall Percentage: 81.33%
Overall Grade: 'A'
=================================================
```

Programming Fundamentals

CT-175

IMPROVEMENT IN CODE

Although the program works correctly, there are several areas where it can be improved for better reliability and flexibility. The input function for the student name and subject name can be changed from scanf to fgets to allow multi-word names and avoid input-related issues. Additionally, validation can be added for the number of subjects to prevent invalid array size allocation. The program could also include a pass or fail status based on overall performance for clearer evaluation. Moreover, the data can be stored in external files to keep student records, which would make the application more practical. These improvements would enhance the program’s usability, stability, and scalability.

Conclusion

- This program successfully calculates and displays a student's academic performance in a clear manner. It uses basic C programming concepts such as loops, arrays, and functions effectively.The structured output makes the result easy to understand.
- The program reduces manual calculation errors by automating percentage and grade evaluation. It allows flexibility in handling different numbers of subjects entered by the user. This makes it useful for academic result processing.
- Overall, the program is simple, efficient, and easy to maintain or modify. It provides a strong foundation for further enhancements such as file storage or graphical output. Thus, it demonstrates a practical application of fundamental programming skills.